FOR COMPLETE BEGINNERS

1.10 + 2.20 = 3.3000000000000003?

Complete Modern Java, episode 8: decimals are not exact, in Java and TypeScript

> A double keeps the nearest binary value. For money, use whole cents or BigDecimal from text.

Bread $1.10 plus milk $2.20 printed 3.3000000000000003, and 0.1 + 0.2 == 0.3 printed false.

In binary, one tenth never ends, so a double stores the nearest value. Two tiny gaps add up.

In this episode

- Why 0.1 is not exact
- The proof with BigDecimal
- Money in whole cents
- BigDecimal from text

Coming next

- Number limits (episode 9)
- Division by zero (episode 9)
- Comparisons (decisions, episode 10)

At the time of recording: Java 25, Node 24 LTS, IntelliJ IDEA 2026.1.3.

01 What each line printed

1.10 + 2.20	`3.3000000000000003`
0.1 + 0.2 == 0.3	`false`
new BigDecimal(0.1)	`0.1000000000000000055511151231257827021181583404541015625`
new BigDecimal("0.1")	`0.1`
110 + 220 (cents, int)	`330`
BigDecimal "1.10" add "2.20"	`3.30`

Every row was run, not predicted.

02 Two ways to keep money exact

Whole cents in an int: whole numbers are stored exactly. Dividing, rounding and the int limit still need care.

BigDecimal made from text, added with .add. Never make it from a double: the double is already rounded.

Money.java

```
import java.math.BigDecimal;

BigDecimal bread = new BigDecimal("1.10");
BigDecimal milk = new BigDecimal("2.20");
BigDecimal total = bread.add(milk);   // 3.30

int totalCents = 110 + 220;           // 330
```

> Printed 3.30 and 330.
>
> Run on a Mac, 2026-10-24: Java 25.0.4.1, Node 24.22.1, TypeScript 7.0.2.

03 TypeScript

number is the same kind of binary double. No built-in decimal type: real projects use whole cents or a decimal library.

money.ts

```
console.log(0.1 + 0.2);               // 0.30000000000000004
console.log(1.10 + 2.20);             // 3.3000000000000003
const breadCents: number = 110;
const milkCents: number = 220;
console.log(breadCents + milkCents);  // 330
```

> npx tsc --noEmit was quiet; node printed the three lines above.
>
> Run on a Mac, 2026-10-24: Java 25.0.4.1, Node 24.22.1, TypeScript 7.0.2.

Money in code

1. Never trust the last digits of a double for money.
2. Whole cents in an int: exact for adding.
3. Or BigDecimal from text, with .add.
4. Comparing decimal results with ==? Take care.
5. Challenge: predict 0.1 + 0.2, BigDecimal "0.1" add "0.2", and 110 + 220.

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The code and the challenge files: complete-modern-java
https://github.com/code-with-sam-dev/complete-modern-java

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. TypeScript is a trademark of Microsoft. IntelliJ IDEA is a trademark of JetBrains. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-24 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.